AICloudStrategist

Reddit Posts — Drafts for Community Seeding

Rule: Reddit hates self-promotion. Lead with value, earn the link. Post on Saturday or Sunday morning IST for best US + India coverage.

Account requirements: - 30+ days old - Active in the subreddit with organic comments before posting - No pattern of spam-dropping blog links - Use Rajiv's or Anushka's personal Reddit — NOT a new account

---

Post 1 — r/aws

Title: The RI coverage gap: engineering teams think it's 70%, the billing data says 28%

Subreddit: r/aws (300K subs)

Body:

TL;DR: In every mid-market AWS audit I've run in the last 6 months, the engineering lead confidently states Reserved Instance coverage is 70%+. Every single time, the AWS Cost Explorer report shows sub-30%. This is the single most expensive blind spot I keep running into — sharing the pattern in case it saves someone a quarterly review.

I've been auditing Indian mid-market AWS accounts (5L–50L/month = roughly $6K–$60K/month in USD) for the last 6 months. The conversation goes almost word-for-word the same every time:

Me: "What's your RI coverage?"
Eng lead: "High. Most of the fleet. Probably 70%+."
Me: (pulls the report)
AWS: "28%."

Why the gap matters: on a $30K/month account, going from 28% to 70% RI coverage on stable workloads is typically $8K–$12K/month saved. For the year that's $100K+. No architectural change, no migration, just a commitment portfolio rebalance.

Why it keeps happening — three patterns:

1. The "we bought reservations" memory is from 2-3 years ago. Those RIs expired. Nobody tracked the expiry. The fleet grew; the commitment didn't.

2. Savings Plan coverage gets confused with Reserved Instance coverage. SPs cover compute broadly but apply differently — an account can have high SP coverage and low RI coverage simultaneously, and the "effective discount rate" question gets answered wrong.

3. Reporting is split between engineering and finance. Engineering looks at instance fleet stability. Finance looks at the bill. Nobody owns the intersection of "what's stable × what's committed × at what effective rate."

How to diagnose in 10 minutes:

1. Go to AWS Cost Explorer → "Reservations" → "Utilization report"
2. Filter last 30 days
3. Look at the "Net RI savings" number vs. your total compute spend
4. If it's below 40% — you have a governance problem, not a technical problem

The fix is almost always a one-afternoon conversation between engineering and finance about who signs off on commitment purchases. Not a platform migration.

Happy to answer questions in comments. I wrote up the full governance framework + measurement approach here

(fair warning: it's on my company site, but no gate, no form): https://aicloudstrategist.com/blog/ri-coverage-india-governance.html

Engagement plan: - Post Sunday 8am IST (= 10:30pm ET Saturday, morning EU, prime-time India) - Answer every comment within 2 hours for the first 24h - Don't correct pedantic responses — let other commenters do that - If someone asks about our services in the comments, link to /book.html — but let them bring it up first

---

Post 2 — r/devops

Title: DORA metrics translated for your CFO — a worked INR example that actually got a DevOps team funded

Subreddit: r/devops (220K subs)

Body:

Engineering teams at Indian mid-market companies keep asking me the same question: "Our CFO asked us to justify DevOps investment. They don't speak DORA. How do we translate?"

Sharing a worked example that actually moved from "proposal" to "approved budget" last year.

Four DORA metrics:
- Deployment frequency
- Lead time for changes
- Change failure rate
- Mean time to restore (MTTR)

Here's the translation CFOs actually respond to:

Deployment frequency: How many releases per week? If it's less than one, there's a deferred-revenue cost — features sitting in staging aren't earning. For a 50Cr ARR business with 20% of revenue dependent on feature velocity, weekly-vs-monthly releases is roughly 10Cr/year of opportunity cost.

Lead time for changes: Hours/days from commit to prod. Translates to engineering payroll waste. A 40-engineer team at avg 25L/year with 30% of their time stuck in release process = 300L/year = 3Cr/year of engineering capacity locked in friction.

Change failure rate: % of releases causing incident. Translates to downtime cost. For an e-comm business at 2L/minute of transaction volume, 2 incidents/month × 45 min MTTR × 2L/min = 180L/year in downtime revenue loss.

MTTR: Average resolution time. Directly maps to the downtime math above.

Add those up for a typical "Low-tier" DORA team: 37L/year in hidden cost that a 20L DevOps platform investment recovers.

The pitch shifts from "we want better tooling" to "here's 37L in hidden cost and here's a 20L investment that recovers it." CFOs sign that kind of paper.

Full worked example with the INR numbers and the conversation script: https://aicloudstrategist.com/blog/dora-metrics-for-cfo.html

Obvious bias disclosure: I consult on exactly this, but the math above is the math above. The blog post has the spreadsheet template linked.

Happy to answer questions on the translation — especially if your CFO is hardcore finance-background and rejecting vague-ROI pitches.

Post 3 — r/startups_IN (r/startups_india)

Title: Your AWS bill is growing 30-40% YoY while revenue grows 15-20%. Here's why, and the three phases every mid-market goes through.

Subreddit: r/startups_india (30K subs, high founder concentration)

Body:

Pattern I've seen in nearly every mid-market Indian tech company I've advised: cloud bill growing ~30-40% YoY, revenue growing 15-20% YoY. The gap is real, it compounds, and most teams don't see it until the CFO escalates.

Most teams move through three phases:

Phase 1 — Over-provisioning (Year 1-2)
Product-market fit is the only priority. Engineers provision generously to avoid outage. EC2 m5.2xlarge where m5.large would do. ElastiCache cluster at 10× anticipated load. Nobody is wrong to do this — founder survival > optimisation.

Phase 2 — Awareness (Year 2-3)
CFO asks "why is the bill 2x our projection." Engineering gets reactive — turns off a few dev environments, maybe buys some Reserved Instances in a panic. Bill flattens for a quarter, then starts growing again.

Phase 3 — Governance (Year 3+)
Company realises cost is an ongoing discipline, not a project. Named ownership, monthly review, commitment portfolio management, tagging taxonomy. Bill growth decouples from arbitrary architectural decisions and actually tracks workload growth.

The critical transition is Phase 2 → Phase 3. Most companies never make it. They stay in "panic → brief flatline → growth" loops forever, and the cumulative overspend becomes a material P&L line.

Why it's hard: the transition requires a coordination conversation (who signs off on commitment purchases — finance or engineering?) that nobody wants to own. It's not a technical problem. It's a role-clarity problem dressed up as a cloud problem.

Full write-up with the governance framework that makes the jump: https://aicloudstrategist.com/blog/indian-cloud-gap.html

For context: I run a FinOps consultancy focused on this transition. But the framework above works whether you hire anyone or not. Shouting into the void here because this pattern is so common and so expensive.

Post 4 — Indie Hackers

Title: Most consultancies can't offer gain-share. Here's why — and why it matters for indie founders.

Venue: indiehackers.com

Body:

If you're running a small SaaS or services business and you've priced a consulting offer in the last year, you've probably felt the same friction I have — clients want outcomes, consultants charge hours, the commercial relationship is misaligned from day one.

I've been experimenting with gain-share as the primary commercial structure for my consultancy (FinOps / cloud cost). Five months in, sharing what's actually been working + why this is hard for Big-4 to replicate.

The structure:
- Small fixed fee covers the engagement work (pays my time + reviewer's time at cost)
- Gain-share clause on top: X% of verified savings above a floor, capped at some multiple
- Measurement window is 12 months so I can't cherry-pick month 1 and run
- Floor is set so the client is protected if nothing dramatic surfaces
- Cap is set so I can't accidentally 10x-bill on a one-time finding

Why it works:
- Client psychologically over the hump: "if you don't find savings, I paid almost nothing"

- My financial upside is directly tied to their verified outcome
- Conversations at scope-negotiation stop being about hours — they're about "what's the biggest outcome we can surface"

Why Big-4 can't do this (and this matters for us indies):
- Their revenue recognition rules require hours × rate
- Their partner compensation is measured on bookings, not verified outcomes
- Their practice-sizing economics require 60%+ utilisation — waiting 3-6 months to invoice gain-share kills the bench math
- Their due-diligence overhead for a gain-share contract (legal review per engagement) exceeds the engagement size

Which means — structurally — this is an indie's pricing lever. A solo consultant or boutique practice can offer gain-share as a differentiator that the big firms literally cannot match without blowing up their internal economics.

Full structure + contract template + measurement approach I use: https://aicloudstrategist.com/blog/gainshare-structure.html

Curious if others here have tried gain-share, outcome-based, or success-fee pricing — what worked, what failed, what's the actual close-rate lift vs. time-and-materials?

After posting — follow-up discipline

First 24 hours: - Check the thread every 30 minutes - Reply to every question/comment (even the critical ones — especially those) - If a thread gets 50+ upvotes, it’s working; double down on engagement - If a thread gets 0-5 upvotes in the first hour, it’s not going to land — don’t force it, move on

If a thread lands (1K+ views): - Monitor for DMs on Reddit — people will reach out - Drop the /book.html link only if directly asked (“how do I work with you”) - Add the top commenters as Twitter follows / send them a personal DM thanking them - Save the thread URL for future use (reference in other contexts)

If a thread doesn’t land: - Don’t delete it — looks desperate - Don’t post the same angle on another subreddit the same week - Wait 2-3 weeks before the next post on that subreddit - Review: was the title too pitchy? Was the first paragraph value or pitch? Did I seed it at the wrong hour?

HackerNews variant (longshot, high upside)

If any of the above posts hits 500+ upvotes on Reddit, adapt it and submit to HN as Show HN. Same content, punchier title: - “Indian mid-market cloud bills grow 30-40% YoY vs 15-20% revenue growth — here’s what I’m seeing” - “Why gain-share pricing is structurally impossible for Big-4 consultancies”

HN lands rarely but when it does, traffic is 10-100x Reddit.

Tracking

For each post: create a Vikunja task in Sprint project with: - Subreddit + title - Posted at (datetime) - Upvote count at 24h, 48h, 7d - DMs received - Discovery calls booked from this post

```
vk add 7 "Reddit r/aws post — RI coverage blind spot" "Posted {date}. Track upvotes, DMs, conversions."
vk label <task_id> rajiv-owns
```